```
        ResourceExecutedContext context) { }            ❹
}
```

❶ Defines whether the feature is enabled

❷ Executes before model binding, early in the filter pipeline

❸ If the feature isn't enabled, short-circuits the pipeline by setting the context.Result property

❹ Must be implemented to satisfy IResourceFilter, but not needed in this case

This simple resource filter demonstrates a few important concepts, which are applicable to most filter types:

- The filter is an attribute as well as a filter. This lets you decorate your controller, action methods, and Razor Pages with it using `[FeatureEnabled(IsEnabled = true)]`.
- The filter interface consists of two methods: `*Executing`, which runs before model binding, and `*Executed`, which runs after the result has executed. You must implement both, even if you only need one for your use case.
- The filter execution methods provide a `context` object. This provides access to, among other things, the `HttpContext` for the request and metadata about the action method or Razor Page that was selected.
- To short-circuit the pipeline, set the `context.Result` property to an `IActionResult` instance. The framework will execute this result to generate the response, bypassing any remaining filters in the pipeline and skipping the action method (or page handler) entirely. In this example, if the feature isn't enabled, you bypass the pipeline by returning `BadRequestResult`, which returns a `400` error to the client.